

AGENTPROF: Semantic Profiling for AI Agents

Anonymous submission

Abstract

AI agents increasingly orchestrate long-running, multi-step activities across users, tools, and system resources, so developers need to determine where failures concentrate, which workflows trigger unsafe effects, and which task categories consume the most budget across many executions. Traditional systems profiling aggregates resource use over stable code paths, whereas existing agent tools trace runs and aggregate metadata but do not link additive system effects to responsible actions and fold them into selectable profiles organized by semantic categories. Agent observability needs profiling, not only debugging. To implement our *semantic operation stack model*, AGENTPROF represents activities as uniform *operations*, replaces runtime stacks with *operation stacks*, provides pluggable mechanisms for intent attribution and stack construction, and exports pprof-compatible profiles. Its label-free constructor learns recurring transitions between visible actions. We evaluate AGENTPROF on a 20-task real Codex capture suite, 405 human-staged CodeTraceBench trajectories, three complete public localization benchmarks, 287 OSWorld-Human sessions, and four public cost workloads. Within the declared process/tool scope, AgentSight capture and joining reach 100.0% precision and 96.6% recall, reject 1,629/1,629 concurrent-control effects, and pass 1,520 linked effects to AGENTPROF, which preserves every one during folding. Label-free recurrence raises ordinary B^3 F1 against human stage partitions from 0.541 to 0.649, with secondary token-weighted gains of 0.076–0.085. Semantic profiles improve MAP over raw-action grouping across trajectories containing annotated targets in all three localization benchmarks. On OSWorld-Human, label-free recurrence reaches 0.680 boundary F1 and 0.786 B^3 F1 versus 0.645 and 0.678 for the strongest simple control and 0.739 and 0.816 for the supervised predictor. AGENTPROF processes 27,765 operations in 1.17 s.

Introduction

AI agents (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Xie et al. 2024) orchestrate multi-step activities across a high-level intent layer (prompts, LLM calls, and tool invocations) and a low-level system-effects layer (process spawns and file and network I/O). As agents evolve from short scripts

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

into complex systems, production teams accumulate many trajectories.

To improve agent quality, safety, and cost efficiency, developers need to analyze agent behavior across many trajectories. Which categories of tasks consume the most token budget? Where do failures concentrate across workflows? Which behavioral patterns trigger unsafe system effects? At scale, manual trajectory inspection is slow and expensive (Lù et al. 2025), while per-trajectory LLM judging requires a separate evaluator pass for every run (Lù et al. 2025; Zheng et al. 2023).

Profiling provides the required aggregation method because traditional systems profilers attribute additive resource consumption to stable code paths rather than debug individual executions (Google 2024b; Gregg 2011). Agent trajectories lack the two structures that make this aggregation possible. Responsibility lies in semantic categories such as task intent and tool operation rather than stable function names. Natural-language prompts provide no common identifier for shared intent. Agent events also lack runtime call nesting. A prompt sequence may represent multiple tasks, while one task may belong to a larger task. Thus adapting profiling requires stable semantic identifiers and an attribution hierarchy that does not depend on a runtime stack.

Existing agent tools provide rich tracing, metadata aggregation, and population dashboards (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024; Datadog 2025; Laminar 2025). Some derive hierarchical categories across traces and aggregate latency, cost, or evaluation metrics (LangChain 2026; Datadog 2026). Existing agent tools do not link additive system effects to responsible agent actions and aggregate them by semantic fields into selectable pprof profiles.

Agent observability needs profiling, not only debugging. Profiling transfers by projecting resources onto stable semantic tags in an attribution hierarchy. Our *semantic operation stack model* provides this structure through operations and operation stacks. *Operations* are uniform records with string fields and additive measures that represent prompts, LLM calls, tool invocations, and system effects without type-specific objects. *Operation stacks* replace the runtime call stack by ordering selected fields. Changing those fields attributes the same operations by task category, execution phase, session, or action type (Figure 1).

We implement this model in AGENTPROF, an offline profiler that compiles local agent trajectories into pprof-compatible profiles (Google 2024b) (Figure 2). *Intent attribution* derives stable, low-cardinality tags from natural-language fields via regex rules, a local LLM tagger, or unsupervised clustering, replacing the identity supplied by function names. *Stack construction* uses either selected fields or label-free recurrence over visible cross-session action transitions to replace runtime nesting.

On 20 real Codex tasks, scoped AgentSight (Zheng et al. 2025) capture and joining reach 100.0% precision and 96.6% recall, reject all 1,629 controls, and pass 1,520 effects that AGENTPROF preserves during folding. On all 405 reconstructable failed CodeTraceBench trajectories (Li et al. 2026), label-free recurrence raises ordinary B³ F1 against human stages over raw action from 0.541 to 0.649, a gain of 0.108 with 95% interval [0.087, 0.129]. Secondary token-weighted gains remain 0.076–0.085 under three shared-response allocations (DeMaere and Darling 2016).

Across three complete public workloads, MAP over target-bearing trajectories rises from 0.773 to 0.789, 0.281 to 0.452, and 0.121 to 0.230, respectively (Fan et al. 2026; Wang et al. 2026; Chen et al. 2026a). On 287 OSWorld-Human development sessions (Abhyankar, Qi, and Zhang 2025), label-free recurrence reaches 0.680 boundary F1 and 0.786 B³ F1. Simple controls reach 0.645 and 0.678, whereas a supervised comparator reaches 0.739 and 0.816.

On four public workloads, AGENTPROF profiles 27,765 operations in 1.17 s and 464.5 MiB, adding 18.2% time and 1.3% memory over raw action.

This paper makes three contributions:

1. **Semantic operation stack model.** Uniform operations and query-time stacks supply agent observability’s missing profiling layer (Background and Motivation; Design).
2. **System: AGENTPROF.** Pluggable intent attribution and stack construction produce pprof-compatible profiles (Implementation).
3. **Evaluation.** Semantic profiles preserve joined effects, improve attribution and problem ranking over raw action, and recover task partitions, declared task/action tags, and session-held-out groups; AGENTPROF profiles 27,765 operations in 1.17 seconds (Evaluation).

Background and Motivation

LLMs and AI Agents

A typical agent trajectory interleaves two layers of activity (Yang et al. 2024; OpenAI 2025; Anthropic 2025; Zheng et al. 2025). At the intent layer, an agent receives prompts, issues LLM calls, and invokes tools for code execution, search, and file editing. Tool invocations trigger lower-layer process, file, and network effects (Zheng et al. 2025); trajectories repeat this cycle, and teams accumulate many trajectories.

System Profiling

Profiling aggregates measurements to attribute resources to responsible entities rather than debug one execution. Traditional profilers sample execution, attach each sample to

its call stack, and fold identical stacks (Google 2024b; Gregg 2011). Flame graphs (Gregg 2011) and pprof (Google 2024b) call graphs visualize aggregate cost by width or node size, while Perfetto (Google 2024a) queries trace events and derives new ones. These tools start from stable code identifiers and runtime stacks; pprof can also promote tagroot/tagleaf values to pseudo-frames. Agent profiles must instead derive stable semantic fields and connect them to downstream effects.

Challenges for Agent Profiling

Existing tools (LangChain 2024; Langfuse 2024; Arize AI 2024a; OpenTelemetry 2024) record agent events as per-execution span trees that support single-run debugging. AgentSight (Zheng et al. 2025) bridges intent and system-effect layers by correlating extended Berkeley Packet Filter (eBPF) system events with intercepted LLM traffic, but the resulting recordings still lack aggregate profiling.

Agent profiling faces two challenges. First, responsibility lies in semantic categories such as task intent and workflow phase, not code paths. Standards represent application-supplied span attributes (OpenTelemetry 2024; Arize AI 2024b), but natural-language histories lack stable categories because different prompts or tools can express the same intent.

Second, agent trajectories lack a runtime call hierarchy for semantic attribution. In CPU profiling, the stack attributes malloc’s cost simultaneously to parse, process, and main. A prompt sequence may contain one or several tasks within a larger workflow, yet no runtime mechanism marks its boundaries or attribution granularity. RQ1 evaluates the attribution consequence, while RQ3 evaluates constructed groups and boundaries.

Design Requirements for Agent Profiling

Traditional profiling relies on three mechanisms that the call stack provides automatically. Agent profiling needs all three but must achieve them without a call stack:

D1: Cross-layer resource projection. The profiler must connect resource consumption from file I/O and process spawns to the responsible prompts and LLM calls.

D2: Stable tags for folding. Without stable function names, the profiler must derive short, repeatable tags from natural language before folding.

D3: Hierarchical attribution. Because span trees do not encode semantic responsibility through function-call nesting, the profiler must construct attribution hierarchies from the data.

The next section maps these requirements to AGENTPROF’s design.

Design

The design maps D1–D3 to ingestion, tags, and operation stacks. For D1, source-specific ingestion links each system effect to its intent, and operations preserve inherited semantic fields for projection and folding. For D2, intent attribution derives stable tags from natural-language fields. For



Figure 1: Semantic flame graphs of 325 real agent trajectories. Width represents token count (top), summed per-operation elapsed duration (middle), and the number of file operations (bottom). All three aggregate the same operation corpus and differ only in which operations they include, which fields form the stack, and which additive measure determines width.

D3, *operation stacks* replace runtime stacks with query-time field projections, supporting multiple attribution granularities over the same data.

The profiling pipeline has four stages: (1) parse raw trajectories into operations, (2) enrich operations via intent attribution or mapping rules, (3) project each operation onto a stack frame sequence chosen at query time, and (4) fold operations with identical stacks, summing their weights.

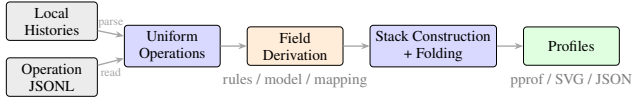


Figure 2: AGENTPROF pipeline. Local histories enter through native history parsing, whereas AgentSight adapters and public datasets enter as newline-delimited JSON operation records. All yield uniform operations before field derivation, stack construction, folding, and profile export.

Semantic Operation Stack Model

AGENTPROF uses one data representation for the intent and system-effect layers described in Background and Motivation. AGENTPROF therefore represents every activity as an operation, a weighted record with string fields and additive measures. Each operation carries string fields (project, agent, session, prompt_tag, kind, model, path, domain, status, ...) and additive measures (token count, duration, event count). The same schema represents prompts, LLM calls, tool invocations, file reads, GUI clicks, and process events. Populated fields distinguish these operation types. When a tool invocation triggers system effects, ingestion propagates its semantic fields to the resulting operations, allowing downstream resource weights

to fold under the responsible intent.

An operation stack provides hierarchical attribution for agent trajectories, replacing the runtime call stack. In CPU profiling, the call stack determines which code paths share responsibility for a resource cost. An operation stack serves the same role. An ordered list of fields $[f_1, f_2, \dots, f_k]$ projects each operation o to a frame sequence $\langle o.f_1, o.f_2, \dots, o.f_k \rangle$, and operations with identical sequences are merged, summing their weights. Unlike a call stack, the fields are chosen at query time, not determined by execution. The same operations can therefore support a flat task summary, a per-session tree, a semantic phase/action hierarchy, or a trajectory organized by the dataset’s own annotations. Changing the fields changes attribution without changing the data. Sessions and spans are optional fields, not hierarchy levels. The same data supports debugging views with `session` and aggregate profiling views without that field.

We formalize each selectable profile as a view (φ, σ, w) . For operation o , the predicate $\varphi(o)$ decides inclusion, an ordered field list $[f_1, \dots, f_k]$ defines $\sigma(o) = \langle o.f_1, \dots, o.f_k \rangle$, and $w(o) \geq 0$ supplies its weight. Users choose the stack function independently of the predicate and weight function. Four built-in resource choices define φ and w for common questions: (1) tokens weights LLM calls by token count, (2) time weights timed operations by duration, (3) files weights file operations by event count, and (4) network weights network operations by event count.

Attribution and Stack Construction

Intent attribution. Intent attribution maps natural-language fields to short, stable, repeatable tags, giving agent trajectories the stable identifiers that traditional profiling derives from function names, as discussed in Background and

Motivation. The framework supports three pluggable backends. Explicit rules provide repeatable user control, a local model tags natural-language fields, and unsupervised grouping discovers candidate categories that developers can encode as rules. Structured trajectories use the same interface to derive fields from existing action and quality attributes. All backends emit ordinary operation fields, keeping the stack model independent of how a tag was obtained.

Stack construction. Stack construction builds attribution hierarchies from available fields. When users supply an ordered list of fields, the projection is direct. When users enable automatic construction, AGENTPROF induces operation stacks without a user-specified field list. We call this constructor *label-free recurrence*. Visible actions are values of the action field available before any target annotation is read. The constructor learns which adjacent visible actions reliably recur across reference sessions. Weak or unseen transitions start a new segment, whereas recurring transitions continue the current segment. The segment’s run-length-compressed action sequence becomes a reusable stack-frame value. The derived group and stack-frame values are ordinary operation fields, so the stack function can compose them with task, phase, action, or session fields at query time. The constructor can learn from a separate label-free operation corpus, so it does not use the target session’s problem annotations.

Implementation

AGENTPROF implements the semantic operation stack model as an offline Rust command-line interface (about 9.8K lines, Figure 2).

Input reconstruction. AGENTPROF reads Codex and Claude Code local JSON Lines (JSONL) histories and operation JSONL. For native histories, it reconstructs prompts, LLM calls, and tool invocations. Because the current CLI does not read AgentSight (Zheng et al. 2025) recordings directly, those recordings pass through a source-specific adapter. For this path, the declared process/tool scope contains the captured target agent process and its launched tool command, excluding concurrent control activity. AgentSight joins system effects within this scope, and the adapter emits operations carrying those links. AGENTPROF then projects and folds their fields and weights.

Field derivation. Regex rules are the production default. Each rule has the form `KIND:TAG=REGEX`, and rules use first-match semantics. Developers can refine the rules by rerunning AGENTPROF and inspecting unmatched operations. Through llama.cpp (Gerganov and contributors 2026), a local LLM tagger can run a quantized 3B-parameter model. The tagger either generates a single-word tag or assigns one tag from a user-declared set of task tags. Both tagging modes use grammar-constrained decoding and caching. TF-IDF (Salton and Buckley 1988) and K-Means (MacQueen 1967) discover candidate prompt groups without predefined rules, while mapping rules derive fields for structured trajectories, for example `phase:navigate=type:(click|scroll|key)`.

Boundary construction. The Rust inducer counts adjacent action transitions in reference sessions and scores each transition by normalized pointwise mutual information (NPMI), $NPMI(a, b) = \ln[p(a, b)/(p_L(a)p_R(b))]/-\ln p(a, b)$, with all three probabilities defined over the same transition sample space (Bouma 2009).

Occurrence-weighted one-dimensional k -means with $k = 2$ (MacQueen 1967) starts at the minimum and maximum scores, assigns distance ties to the lower center, and uses the converged midpoint as a global cutoff. The inducer repeats this procedure over transitions between different actions to obtain a second cutoff. Same-action pairs use the global cutoff, whereas pairs with different actions use the smaller cutoff. The second cutoff can therefore remove but never add a boundary from the global rule. Unseen transitions or scores below the applied cutoff start a new segment. Each segment’s run-length-compressed action sequence becomes its stack-frame value. The inducer reads only session identity, input order, and action. Target annotations and resource weights do not affect construction.

An optional reference-calibrated recurrence mode keeps the NPMI score but selects one scalar cutoff that maximizes per-operation B³ F1 on disjoint reference groups. The mode enumerates score intervals, resolves exact ties toward the smallest cutoff, and never reads target groups. Label-free k -means remains the default.

Profile export. Output formats are pprof protobuf for `go tool pprof -http`, folded stacks for `flamegraph.pl`, standalone SVG flame graphs, and JSON.

Evaluation

Research questions. The evaluation addresses four research questions: **RQ1:** Does semantic profiling improve resource attribution? **RQ2:** Does profiler output correspond to real problems? **RQ3:** How accurate are the tags? **RQ4:** What is the profiling cost? Each RQ subsection pairs its protocol with the strongest answer its evidence supports.

We use three classes of data. **Real inputs (RQ1)** comprise 325 local histories (198 Codex, 50 Claude, and 77 Claude subagent) with 183,714 system observations collected over multiple months, plus a separate 20-task Codex capture suite with concurrent controls.

Annotated public trajectories (RQ1, RQ3, and RQ4) comprise 15 real agent or human execution families converted by mapping rules into 47,590 operations, plus a separate 405-trajectory, 20,866-operation CodeTraceBench stage population. They span web, API, coding, and mobile/GUI agents (Lù, Kasner, and Reddy 2024; Deng et al. 2023; Xu et al. 2025; Yao et al. 2022; Li et al. 2023; Qin et al. 2024; Yao et al. 2025; Yang et al. 2024; Li et al. 2024; Lu et al. 2025; Liu et al. 2026). Four families converted by mapping rules provide structured quality or boundary annotations, including AgentRewardBench (Lù et al. 2025), SATraj-OS (Chen et al. 2026b), AgentNet (Wang et al. 2025), and OSWorld-Human (Abhyankar, Qi, and Zhang 2025).

Problem-localization benchmarks (RQ2) are AgentProcessBench, HINTBench, and TraceElephant, which provide

independent released targets over 27,346 operations (Fan et al. 2026; Wang et al. 2026; Chen et al. 2026a). The 27,765-operation RQ4 union is drawn from four public workloads. These experiment-specific totals are not pooled.

RQ1: Resource Attribution

RQ1 asks whether semantic profiling improves resource attribution beyond what traditional tag-free tools provide. We first test whether the capture-and-join path assigns system effects within the declared process/tool scope while rejecting concurrent controls. We use 20 real Codex tasks, each paired with one concurrent negative control. AgentSight 0.2.37 captures each pair and joins effects within that scope. An adapter converts the joined effects to operation JSONL, which `agentpprof 0.2.37` folds.

All 20 tasks and their controls completed. The capture-and-join path recovers 1,520 of 1,574 in-scope effects with no false positives (100.0% precision, 96.569% recall), and none of 1,629 control effects joins. AGENTPROF emits 1,520 samples in 152 stacks, exactly preserving the input total and all five predeclared task-category totals.

This result establishes capture-and-join precision and recall within the suite’s declared process/tool scope, and loss-less folding by AGENTPROF. The task categories are predeclared rather than automatically inferred.

We next test whether the current operation stack assigns measured LLM resources to independently annotated units of work more faithfully than raw operation identity. We use all 405 failed CodeTraceBench trajectories whose released sources yield the official operation sequence. The population contains 20,866 operations and 2,948 human-verified contiguous stage intervals across OpenHands, mini-SWE-agent, SWE-agent, and Terminus2 (Li et al. 2026). We use the same recurrence constructor as in RQ3. It reads neither target stages nor resource weights. Matched raw-action grouping starts a new group whenever the source action field changes. Phase-only and action-kind grouping use the named source field, while session grouping places each session in one group. Per-operation grouping makes each operation a singleton. Every method receives the same operations, stages, and provider-reported token measurements.

We use ordinary operation-level B^3 precision, recall, and F1 as the primary metric for partition agreement (Bagga and Baldwin 1998). We also report token-weighted B^3 (DeMaere and Darling 2016) only as a resource-sensitive secondary analysis.

Method	B^3 P	B^3 R	B^3 F1	Token-wt.
				B^3 F1
Label-free recurrence	0.829	0.534	0.649	0.651
Raw-action grouping	0.891	0.388	0.541	0.567
Phase only	0.686	0.626	0.654	0.652
Action kind	0.948	0.315	0.473	0.476
Per session	0.174	1.000	0.296	0.323
Per operation	1.000	0.141	0.248	0.257

Table 1: RQ1 attribution agreement on 405 CodeTraceBench trajectories. Ordinary B^3 is the standard partition-agreement metric. Token-weighted B^3 F1 is a resource-weighted secondary analysis under equal allocation.

Table 1 shows that recurrence raises ordinary B^3 F1 over raw action identity from 0.541 to 0.649. The gain is 0.108 with a paired 95% bootstrap interval clustered by task of [0.087, 0.129], and is positive in all four agent frameworks.

The adapter maps 494.9 million tokens from 17,148 responses that contain operations. Of these, 1,426 responses contain multiple operations. Allocating each shared response equally, entirely to its first operation, or entirely to its last operation leaves the recurrence-minus-raw weighted F1 gain positive at 0.085, 0.076, and 0.076, respectively. The total mapped token count is conserved exactly.

Phase-only reaches a statistically indistinguishable 0.654 ordinary B^3 F1. This experiment therefore supports attribution by semantic stages over raw action identity, but it does not establish one semantic profile as universally dominant. RQ3 separately tests the recurrence constructor against phase-only boundaries on OSWorld-Human, where recurrence has a larger advantage. Because this population influenced constructor selection, the result is post-hoc support rather than independent confirmation.

As a separate use case, five declared field hierarchies project the same 13,265 operations from nine trajectory families. The resulting profiles contain 9 dataset, 57 phase, 226 semantic, 455 action, or 3,757 per-session groups. The families are WebLINX, WebShop, APiBank, AgentTrek, SWE-agent, Mind2Web, ToolBench, AndroidControl, and GUI Odyssey.

The semantic hierarchy adds task, operation type, tool, and status to dataset and phase. The five depths compare budgets by task, repeated actions across sessions, and individual-session details. The same model covers all 15 trajectory families without type-specific objects.

The `time` and `tokens` views of the same 325 trajectories share only 7/10 top prompt categories (Spearman $\rho = 0.623$), and `network`-tagged prompts rank 8th by time but 93rd by tokens. Thus, multiple weights expose bottlenecks that one metric would miss.

Figure 1 shows the resulting `tokens`, `time`, and `files` flame graphs, illustrating that changing only the selection predicate, stack fields, and weight function produces different aggregates from the same input operation corpus. Together, these results answer RQ1. The capture-and-join path rejects concurrent controls, AGENTPROF preserves attributed weight, and standard partition agreement on human stage intervals shows that semantic attribution improves over raw action identity. The recurrence advantage over raw-action

grouping remains positive under all three shared-response token allocations.

Selectable stack depths and weights then expose different responsibility and bottleneck views from the same operations. RQ3 separately evaluates tag and group construction accuracy.

RQ2: Problem Correspondence

RQ2 asks whether target-blind profiles place independently annotated problems in groups a developer can inspect early. We run complete AgentProcessBench, HINTBench, and TraceElephant workloads (Fan et al. 2026; Wang et al. 2026; Chen et al. 2026a). Of HINTBench’s reported 629 trajectories, the released test snapshot used here contains 536. A separate 80-trajectory validation snapshot selects one of 24 field orders before all 536 tests are scored.

Within each workload, AGENTPROF and raw-action grouping use identical operations and the same benchmark-provided judge votes or localizer hits. The two methods differ only in grouping. AgentProcessBench averages judge votes within each group. For HINTBench and TraceElephant, every root-to-frame stack prefix receives the 95% Wilson lower bound (Wilson 1927) of its member hits, and an operation inherits the maximum score among prefixes containing it, with ties preserved.

We choose profiles, fields, and scoring rules before revealing independent test targets. Each query is one trajectory containing an annotated target. We report non-interpolated average precision (AP) per query and mean average precision (MAP) across queries.

Workload	Queries	AGENTPROF	Raw action	Δ [95% CI]
AgentProcessBench	614	.789	.773	+.016 [.005,.027]
HINTBench	400	.452	.281	+.171 [.154,.188]
TraceElephant	220	.230	.121	+.109 [.077,.142]

Table 2: MAP over queries after scoring every operation in each complete workload. Each query is one trajectory containing an annotated target. Higher is better. Intervals use 10,000 paired resamples of trajectory clusters within benchmark-defined strata.

AGENTPROF improves MAP over raw action on all three complete workloads (Table 2). Thus the standard primary metric answers RQ2 positively against the matched raw-action organization.

We then perform a post-hoc mechanism analysis on these same populations. A local-first ranking preserves every strict operation-local score ordering and uses semantic recurrence only to refine exact local-score ties. Its MAP is .896/.545/.322, compared with .863/.411/.209 for local-only and .789/.452/.230 for semantic-only ranking. Against an identically composed local-plus-raw tie-breaker, the gain is +.039 [.029,.049] on HINTBench and +.073 [.050,.097] on TraceElephant; AgentProcessBench does not distinguish the two refinements (+.003 [-.0005,.0069]). This adaptive analysis supports a simple mechanism—semantic recurrence re-

fines, rather than overrides, local diagnostic evidence—but is not an untouched confirmation.

RQ3: Tag Accuracy

RQ3 evaluates how accurately and consistently predeclared, target-blind taggers and mappings recover independent semantic labels and workflow structure. Named task and action fields are evaluated as literal multiclass predictions; task, phase, and group structure are evaluated as permutation-invariant partitions or exact boundaries against benchmark annotations. Here, a *tag* is an operation field produced by AGENTPROF, a *reference annotation* is benchmark-provided truth, and a predicted group or boundary is structural output derived from tags. We compare predicted task partitions, literal task-family and action tags, and group boundaries with their reference annotations, then fold predicted OSWorld groups with their original additive weights.

Table 1 evaluates the deterministic phase field structurally: phase-only grouping reaches 0.654 ordinary B³ F1 against 2,948 human stages over 20,866 operations. Below we test literal labels, other partitions, and boundaries.

We test all 287 OSWorld-Human task-instance sessions (Abhyankar, Qi, and Zhang 2025) with complete group annotations, comprising 3,978 operations, 3,691 adjacent pairs, and 2,042 groups. For the supervised Bernoulli Naive Bayes predictor (McCallum and Nigam 1998), we fix the model family, nine input fields, adjacent-pair features, and training procedure. Each session-held-out cross-validation fold fits its parameters and threshold on training sessions and predicts every held-out session once. Predicted boundaries become an ordinary group field that AGENTPROF projects and folds through the Semantic Operation Stack Model’s profile-construction path.

With the same five folds, label-free recurrence uses only the other sessions’ visible action transitions. Because we designed the recurrence rule after inspecting earlier results on this corpus, we treat these results as development evidence rather than independent cross-family confirmation. Optional reference-calibrated recurrence uses those training operations and group annotations to fit one NPMI cutoff. Held-out folds contribute only visible actions until predictions are fixed.

Three controls place a boundary after every operation, after a visible action change, or after a visible phase change.

We report exact adjacent-boundary precision, recall, and F1, plus ordinary per-operation B³ partition F1 (Bagga and Baldwin 1998), whose per-operation predicted–human group overlap captures merging and fragmentation.

Method	Prec.	Recall	Boundary F1	B ³ F1
Supervised predictor	0.700	0.782	0.739	0.816
Reference-calibrated recurrence	0.610	0.922	0.734	0.801
Label-free recurrence	0.592	0.799	0.680	0.786
Always boundary	0.476	1.000	0.645	0.678
Action change	0.386	0.626	0.477	0.659
Phase change	0.441	0.268	0.334	0.665

Table 3: RQ3 boundary and partition agreement on OSWorld-Human. Boundary F1 scores exact adjacent-pair decisions. B³ scores predicted-human partitions.

The supervised predictor reaches 0.739 boundary F1 and 0.816 B³ F1 versus 0.645 and 0.678 for the strongest controls. Label-free recurrence reaches 0.680 and 0.786. Reference-calibrated recurrence reaches 0.734 and 0.801 but requires group annotations that the default label-free method does not use. Every method also conserves the total projected weight.

For target-blind TF-IDF/K-Means, V-measure (Rosenberg and Hirschberg 2007) over operation assignments reaches 0.557 on nine Mind2Web sessions (49 operations) and 0.815 on 100 ScienceWorld sessions (2,504 operations) (Deng et al. 2023; Wang et al. 2022). Every operation receives a prediction in both datasets, whereas a constant tag yields V-measure 0. This evaluation tests partitions, not literal names.

For task-family tags, the predeclared Qwen3.6-27B configuration classifies all 1,012 AgentBoard goals across nine families from only goals and declared family descriptions (Qwen Team 2026; Ma et al. 2024). It reaches 0.695 macro-F1 and 0.733 accuracy versus a majority-class baseline at 0.044 and 0.248, with identical three-run assignments.

For action tags, the predeclared Qwen3.6-27B configuration reads only the current thought/action and eight definitions for all 2,737 published annotations from 120 AutoCodeRover, OpenHands, and RepairAgent trajectories (Qwen Team 2026; Bouzenia and Pradel 2025; Sajadi et al. 2026). The model reaches 0.498 macro-F1 and 0.628 accuracy versus a majority-class baseline at 0.061 and 0.323. Its 0.437 macro-F1 gain has a whole-trajectory 95% bootstrap interval of [0.380, 0.494], and both complete runs agree exactly.

Thus, on the named public populations, RQ3 establishes target-blind task partitions at V-measure 0.557 and 0.815, phase structure at 0.654 ordinary B³ F1, task-family assignment at 0.695 macro-F1, action assignment at 0.498 macro-F1, and label-free group boundaries at 0.680 boundary F1 and 0.786 B³ F1.

RQ4: Profiling Cost

RQ4 measures the time to parse fixed-field operation JSONL, construct stacks, fold weights, and serialize the profile. The measurement excludes capture, source adaptation, and field/tag generation. We compare the fixed semantic hierarchy `project, agent, task, phase, op, tool, status` with the raw hierarchy `project, agent, action, status` on identical inputs across three runs on four complete public workloads and their union, using `agentpprof 0.2.37` on a 24-core Intel Core Ultra

9 285K with 125 GiB RAM and Linux 6.15.11.

Workload	Ops	Semantic (s)	Raw action (s)	Peak RSS (MiB)
AgentRewardBench	729	0.04	0.03	19.0
SATraj-OS	4,285	0.17	0.14	73.5
OSWorld-Human	6,010	0.25	0.21	108.4
AgentNet	16,741	0.71	0.58	278.4
Union	27,765	1.17	0.99	464.5

Table 4: RQ4 profile-construction cost. Times are three-run medians, and resident set size (RSS) is the largest peak observed for the semantic profile.

Both time curves are monotonic across the five input sizes. The semantic curve has a descriptive slope of 0.0422 ms per operation ($R^2 = 0.9997$), reaching 23,731 operations/s on the union. RQ4 therefore finds that semantic construction completes in 1.17 seconds at 464.5 MiB for 27,765 operations and adds 180 ms (18.2%) and 6.0 MiB maximum RSS (1.3%) over raw-action grouping.

Scope and Limitations

RQ1 covers 20 fixed tasks under declared process/tool scope and all 405 failed CodeTraceBench trajectories (Li et al. 2026) reconstructable as official operation sequences. Its 494.9 million mapped tokens cover operation-bearing responses, not every released model call. Because this corpus influenced constructor selection, its result is post-hoc support, not independent confirmation.

RQ3 tests CodeTraceBench phase structure, Mind2Web/ScienceWorld partitions, a named backend over nine task families and eight action tags, and session-held-out OSWorld-Human boundaries; these measurements cover the named populations and declared tag sets.

Related Work

Agent observability. Platforms aggregate spans, metadata, and evaluations, while some derive cross-trace categories or profile workflows (LangChain 2024; Langfuse 2024; OpenTelemetry 2024; Datadog 2025; LangChain 2026; NVIDIA 2026). AGENTPROF instead links effects to responsible actions and folds additive measures across histories into selectable pprof profiles.

Systems profiling. pprof and flame graphs fold runtime stacks, while Perfetto queries trace events (Google 2024b; Gregg 2011; Google 2024a). AGENTPROF instead folds query-time semantic stacks without code identities or runtime nesting.

Agent diagnosis. AgentRx diagnoses failures, CodeTracer reconstructs states, and benchmarks score faulty steps or agents (Barke et al. 2026; Li et al. 2026; Fan et al. 2026; Wang et al. 2026; Chen et al. 2026a). AGENTPROF instead profiles recurring categories that accumulate resources or problems across trajectories.

Conclusion

Agent observability needs profiling, not only debugging. AGENTPROF realizes it via cross-run operation stacks.

References

- Abhyankar, R.; Qi, Q.; and Zhang, Y. 2025. OSWorld-Human: Benchmarking the Efficiency of Computer-Use Agents. arXiv preprint arXiv:2506.16042.
- Anthropic. 2025. Claude Code: An Agentic Coding Tool. <https://code.claude.com/docs/en/overview>.
- Arize AI. 2024a. Arize Phoenix. <https://arize.com/docs/phoenix>.
- Arize AI. 2024b. OpenInference Specification. <https://arize-ai.github.io/openinference/spec/>.
- Bagga, A.; and Baldwin, B. 1998. Entity-Based Cross-Document Coreferencing Using the Vector Space Model. In *36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, Volume 1*, 79–85. Montreal, Quebec, Canada: Association for Computational Linguistics.
- Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv preprint arXiv:2602.02475.
- Bouma, G. 2009. Normalized (Pointwise) Mutual Information in Collocation Extraction. In *From Form to Meaning: Processing Texts Automatically, Proceedings of the Biennial GSCL Conference 2009*, 31–40. Tübingen, Germany.
- Bouzenia, I.; and Pradel, M. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *2025 IEEE/ACM 40th International Conference on Automated Software Engineering (ASE)*, 2846–2857. IEEE.
- Chen, M.; Wang, J.; Mu, F.; Wang, Y.; Liu, Z.; Feng, H.; and Wang, Q. 2026a. Seeing the Whole Elephant: A Benchmark for Failure Attribution in LLM-Based Multi-Agent Systems. arXiv preprint arXiv:2604.22708.
- Chen, X.; Yin, Z.; He, S.; Huang, B.; Lei, S.; et al. 2026b. Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. arXiv preprint arXiv:2605.06230.
- Datadog. 2025. LLM Observability. https://docs.datadoghq.com/llm_observability/.
- Datadog. 2026. LLM Observability Patterns. https://docs.datadoghq.com/llm_observability/monitoring/patterns/.
- DeMaere, M. Z.; and Darling, A. E. 2016. Deconvoluting Simulated Metagenomes: The Performance of Hard- and Soft-Clustering Algorithms Applied to Metagenomic Chromosome Conformation Capture (3C). *PeerJ*, 4: e2676.
- Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.
- Fan, S.; Ye, X.; Huo, Y.; Chen, Z.-Y.; Guo, Y.; Yang, S.; Yang, W.; Ye, S.; Chen, J.; Chen, H.; Cong, X.; and Lin, Y. 2026. AgentProcessBench: Diagnosing Step-Level Process Quality in Tool-Using Agents. In *Proceedings of KDD*.
- Gerganov, G.; and contributors. 2026. llama.cpp: LLM Inference in C/C++. GitHub repository.
- Google. 2024a. Perfetto Trace Processor. <https://perfetto.dev/docs/analysis/trace-processor>.
- Google. 2024b. pprof. <https://github.com/google/pprof>.
- Gregg, B. 2011. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- Laminar. 2025. Signals: Structured Event Extraction from Traces. <https://docs.lmn.ai/signals/introduction>.
- LangChain. 2024. LangSmith Observability. <https://docs.langchain.com/langsmith/observability>.
- LangChain. 2026. LangSmith Insights. <https://docs.langchain.com/langsmith/insights>.
- Langfuse. 2024. Langfuse Observability. <https://langfuse.com/docs/observability/overview>.
- Li, H.; Yao, Y.; Zhu, L.; Feng, R.; Ye, H.; Wang, J.; He, Y.; Zou, P.; Zhang, L.; Lei, X.; Huang, H.; Deng, K.; Sun, M.; Zhang, Z.; Ye, H.; and Liu, J. 2026. CodeTracer: Towards Traceable Agent States. arXiv:2604.11641.
- Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116.
- Li, W.; Bishop, W. E.; Li, A.; Rawles, C.; Campbell-Ajala, F.; Tyamagundlu, D.; and Riva, O. 2024. On the Effects of Data Scale on UI Control Agents. In *NeurIPS 2024, Datasets and Benchmarks Track*.
- Liu, Z.; Xie, J.; Ding, Z.; Li, Z.; Yang, B.; et al. 2026. ScaleCUA: Scaling Open-Source Computer Use Agents with Cross-Platform Data. In *Proceedings of ICLR*.
- Lu, Q.; Shao, W.; Liu, Z.; Meng, F.; Li, B.; Chen, B.; Huang, S.; Zhang, K.; Qiao, Y.; and Luo, P. 2025. GUIOdyssey: A Comprehensive Dataset for Cross-App GUI Navigation on Mobile Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.
- Lù, X. H.; Kasner, Z.; and Reddy, S. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. In *Proceedings of ICML, volume 235 of Proceedings of Machine Learning Research*, 33007–33056. PMLR.
- Lù, X. H.; Kazemnejad, A.; Meade, N.; Patel, A.; Shin, D.; Zambrano, A.; Stańczak, K.; Shaw, P.; Pal, C. J.; and Reddy, S. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. In *Conference on Language Modeling (COLM)*.
- Ma, C.; Zhang, J.; Zhu, Z.; Yang, C.; Yang, Y.; Jin, Y.; Lan, Z.; Kong, L.; and He, J. 2024. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *Advances in Neural Information Processing Systems, volume 37*, 74325–74362. Curran Associates, Inc.
- MacQueen, J. B. 1967. Some Methods for Classification and Analysis of Multivariate Observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, volume 1*, 281–297. University of California Press.
- McCallum, A.; and Nigam, K. 1998. A Comparison of Event Models for Naive Bayes Text Classification. In *AAAI-98 Workshop on Learning for Text Categorization*, 41–48. AAAI Press.

- NVIDIA. 2026. Profiling and Performance Monitoring of NVIDIA NeMo Agent Toolkit Workflows. Official documentation.
- OpenAI. 2025. OpenAI Codex CLI. GitHub repository.
- OpenTelemetry. 2024. OpenTelemetry GenAI Semantic Conventions. <https://github.com/open-telemetry/semantic-conventions-genai>.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerber, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.
- Qwen Team. 2026. Qwen3.6-27B: Flagship-Level Coding in a 27B Dense Model. Official model card.
- Rosenberg, A.; and Hirschberg, J. 2007. V-Measure: A Conditional Entropy-Based External Cluster Evaluation Measure. In *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, 410–420. Prague, Czech Republic: Association for Computational Linguistics.
- Sajadi, A.; Nguyen, T.; Huynh, K.; Parra, E.; and Chatterjee, P. 2026. TraceView: Interactive Visualization of Agentic Program Repair Trajectories. arXiv:2606.22110.
- Salton, G.; and Buckley, C. 1988. Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing and Management*, 24(5): 513–523.
- Wang, J.; Hou, J.; Wang, F.; Jian, P.; Bao, C.; and Lv, Z. 2026. HINTBench: Horizon-Agent Intrinsic Non-Attack Trajectory Benchmark. arXiv preprint arXiv:2604.13954.
- Wang, R.; Jansen, P.; Côté, M.-A.; and Ammanabrolu, P. 2022. ScienceWorld: Is Your Agent Smarter than a 5th Grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 11279–11298. Association for Computational Linguistics.
- Wang, X.; Wang, B.; Lu, D.; Yang, J.; Xie, T.; et al. 2025. AgentNet. <https://huggingface.co/datasets/xlangai/AgentNet>.
- Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.
- Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OS-World: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.
- Xu, Y.; Lu, D.; Shen, Z.; Wang, J.; Wang, Z.; Mao, Y.; Xiong, C.; and Yu, T. 2025. AgentTrek: Agent Trajectory Synthesis via Guiding Replay with Web Tutorials. In *Proceedings of ICLR*.
- Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.
- Yao, S.; Chen, H.; Yang, J.; and Narasimhan, K. 2022. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.
- Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *The Thirteenth International Conference on Learning Representations (ICLR)*.
- Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.; Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.
- Zheng, Y.; Hu, Y.; Yu, T.; and Quinn, A. 2025. AgentSight: System-Level Observability for AI Agents Using eBPF. In *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems*.